Programming for AI

Week 3 Activity

Task 1: Encapsulated Model Weight Tracker (Classes, Name Mangling, Getters & Setters)

Goal: In Python, we define member variables inside __init__ and rely on naming conventions and name mangling (__variable) for encapsulation. Getters and setters are used when data validation logic is needed.

The Problem: Write a class named ModelLayer that satisfies the following requirements:

1. **Constructor (__init__):**
   - Accepts layer_name (string), learning_rate (float, default value: 0.01), and initial_weights (list of floats, default value: None).
   - If initial_weights is None, initialize it as an empty list [].
   - Store layer_name as a public attribute.
   - Store learning_rate as a protected attribute _learning_rate.
   - Store the weights as a private attribute __weights.
2. **Getter & Setter:**
   - Create a getter method get_weights() that returns a copy of __weights.
   - Create a setter method set_weights(new_weights) that validates the input:
     - Check if new_weights is a non-empty list of numeric values. If valid, update __weights.
     - If the list is empty or contains non-numeric values, raise a ValueError with the message "Invalid weight values provided".
3. **Verification:**
   - Instantiate an object: layer = ModelLayer("Dense_1", initial_weights=[0.25, -0.15, 0.40]).
   - Attempt direct access via layer.__weights inside a try-except AttributeError block to confirm it is inaccessible directly from the outside.
   - Print the mangled attribute directly using its mangled name layer._ModelLayer__weights to observe Python's name-mangling behavior.

Task 2: Custom Batch Iterator (Iterator Protocol & StopIteration)

Goal: In Python, an iterator is any object that implements __iter__() and __next__(). When reaching the end of the data stream, instead of returning an end-pointer, it raises a StopIteration exception.

The Problem: Write a custom iterator class DatasetBatcher that streams slices of a dataset:

1. **Requirements:**
   - The constructor __init__(self, data, batch_size=2) takes a dataset (list) and an integer batch_size.
   - Maintain an internal index pointer self.current_idx initialized to 0.
   - Implement __iter__(self) to return self.
   - Implement __next__(self):
     - Returns the next sub-list (slice) of size batch_size.
     - If the pointer reaches or exceeds the length of data, raise StopIteration.
2. **Execution:**
   - Given the dataset samples = [101, 102, 103, 104, 105, 106, 107], instantiate batcher = DatasetBatcher(samples, batch_size=3).
   - Manually extract the first two batches using the next() built-in function.
   - Extract the remaining batches using a standard for loop to verify that the StopIteration is caught cleanly.

Task 3: StreaminAg Deep Learning Data Loader (Generators & yield)

Goal: Loading an entire large dataset into RAM at once can crash your system memory. A generator function uses the yield keyword to pause execution, preserve local variable state, and stream records one at a time on demand.

The Problem: Write a generator function stream_sensor_data(signal_readings, threshold=50.0):

1. **Logic:**

- Loop through signal_readings (a list of float values).
- For each reading, check if it strictly exceeds threshold.
- If it exceeds the threshold, yield a tuple containing: (index, reading, "ANOMALY").
- If it does not exceed the threshold, yield a tuple containing: (index, reading, "NORMAL").

2. **Pipeline Chaining:**
   - Write a second generator function filter_anomalies(generator_stream) that accepts the generator from above and only yields tuples marked as "ANOMALY".
3. **Execution:**
   - Test your pipeline with readings = [23.4, 55.1, 48.9, 72.3, 12.0, 99.8, 30.5].
   - Stream the filtered anomalies directly into a for loop, printing each detected anomaly without loading the entire collection into a new static list.

Task 4: API Accessor & Model Config Factory (Closures)

Goal: A closure is a nested function that retains access to variables from its enclosing scope even after the outer function has finished executing. It acts as a lightweight alternative to a class when you only need to encapsulate a single piece of state.

The Problem: Write an outer factory function create_feature_scaler(min_val, max_val):

1. **Requirements:**
   - The outer function accepts min_val and max_val (representing the normalization boundaries of a dataset feature).
   - Define an inner function scale(x) that computes and returns the min-max normalized value using the proper formula:

$$x_{i_{new}} = \frac{x_i - x_{min}}{x_{max} - x_{min}}$$

   - Return the scale inner function object (do not call it inside the outer function).
2. **Execution:**
   - Create two independent scalers:
     - age_scaler = create_feature_scaler(min_val=18, max_val=65)
     - grade_scaler = create_feature_scaler(min_val=0, max_val=20)
   - Verify state persistence:
     - Call age_scaler(30) and grade_scaler(15).
     - Inspect age_scaler.__closure__[0].cell_contents and age_scaler.__closure__[1].cell_contents to inspect the variables preserved inside the closure's memory scope.

Task 5: AI Execution Timer & Input Validator (Decorators)

Background: A decorator is a higher-order function that takes another function as an argument, wraps it with additional behavior, and returns the modified function. The @ symbol provides syntactic sugar for func = decorator(func).

The Problem: Write a decorator validate_and_time to act as an execution gatekeeper:

1. **Requirements:**
   - The decorator must accept any function func.
   - Inside the wrapper function (*args, **kwargs):
     - **Validation Check:** Inspect all arguments passed to func. If any numeric argument is negative (<0), do not execute func; instead, raise a ValueError("Negative input parameters are not allowed.").
     - **Timing:** Record the start time using time.perf_counter().
     - **Execution:** Call func(*args, **kwargs) and store the result.
     - **Logging:** Record the end time, print the elapsed runtime in milliseconds, and return the function's result.
2. **Application:**
   - Apply @validate_and_time to an AI loss calculation function:
   - Define a fucntion compute_batch_loss(predictions, targets) which implements the following formula:

$$\frac{1}{n}\sum_{i=1}^{n}(pred_i - target_i)^2$$

3. **Execution:**
   - Call compute_batch_loss([1.2, 2.5, 3.1], [1.0, 2.0, 3.0]) to observe the timing output.
   - Call compute_batch_loss([-1.0, 2.0], [1.0, 2.0]) inside a try-except block to confirm that the input validator blocks the execution and triggers the ValueError.